RV32IMA System-on-Chip Datasheet

RV32 Development Team

Version 1.0, 2026-02-25

Table of Contents

1. Abstract . . . 1
2. Introduction . . . 2
   2.1. Overview . . . 2
   2.2. Key Features . . . 2
   2.3. Document Organization . . . 2
3. System Architecture . . . 4
   3.1. Block Diagram . . . 4
   3.2. System Components . . . 5
      3.2.1. Processor Core (rv32_core) . . . 5
      3.2.2. Instruction Cache (rv32_icache) . . . 6
      3.2.3. Memory Subsystem . . . 6
      3.2.4. Interrupt System . . . 6
4. Memory Map . . . 7
   4.1. Address Space Overview . . . 7
   4.2. Address Decode Rules . . . 7
5. Processor Core Specification . . . 9
   5.1. RV32IMA ISA Support . . . 9
   5.2. Pipeline Architecture . . . 9
      5.2.1. Instruction Buffer (IB) . . . 9
      5.2.2. Store Buffer (SB) . . . 10
      5.2.3. Forwarding and Hazards . . . 10
   5.3. Performance Characteristics . . . 10
   5.4. CSR (Control and Status Registers) . . . 10
   5.5. Exception and Interrupt Handling . . . 11
      5.5.1. Exception Types . . . 11
      5.5.2. Interrupt Types . . . 11
      5.5.3. Trap Handling . . . 12
6. Instruction Cache Specification . . . 13
   6.1. Overview . . . 13
   6.2. Cache Organization . . . 13
   6.3. Cache Operations . . . 13
      6.3.1. Normal Fetch (Cache Hit) . . . 13
      6.3.2. Cache Miss . . . 14
      6.3.3. Bypass Mode . . . 14
   6.4. Cache Management Operations (CMO) . . . 14
   6.5. Performance Counters (Simulation Only) . . . 14
7. Peripheral Specifications . . . 15
   7.1. UART (Universal Asynchronous Receiver/Transmitter) . . . 15

7.1.1. Overview . . . 15
7.1.2. Register Map . . . 15
7.1.3. Baud Rate Configuration . . . 16
7.1.4. Interrupt Behavior . . . 16
7.1.5. External Pins . . . 16
7.2. SPI (Serial Peripheral Interface) . . . 17
7.2.1. Overview . . . 17
7.2.2. Register Map . . . 17
7.2.3. SPI Modes . . . 18
7.2.4. Clock Divider Examples (100 MHz System Clock) . . . 18
7.2.5. Usage Flow . . . 18
7.2.6. External Pins . . . 19
7.3. I2C (Inter-Integrated Circuit) . . . 19
7.3.1. Overview . . . 19
7.3.2. Register Map . . . 19
7.3.3. Clock Divider Examples (100 MHz System Clock) . . . 20
7.3.4. Usage Flow . . . 20
Write Transaction . . . 20
Read Transaction . . . 21
7.3.5. External Pins . . . 21
7.4. DMA (Direct Memory Access Controller) . . . 21
7.4.1. Overview . . . 21
7.4.2. Register Map . . . 22
7.4.3. Transfer Modes . . . 23
1D Transfer (Linear) . . . 23
2D Transfer (Strided) . . . 23
3D Transfer (Planar) . . . 24
Scatter-Gather Transfer . . . 24
7.4.4. Interrupt Behavior . . . 24
7.4.5. Performance Monitoring . . . 24
7.4.6. AXI Interface . . . 24
7.5. CLINT (Core-Local Interruptor) . . . 25
7.5.1. Overview . . . 25
7.5.2. Register Map . . . 25
7.5.3. Timer Operation . . . 25
7.5.4. Software Interrupt Operation . . . 26
7.6. PLIC (Platform-Level Interrupt Controller) . . . 26
7.6.1. Overview . . . 26
7.6.2. Register Map . . . 26
7.6.3. Interrupt Sources . . . 27
7.6.4. Interrupt Flow . . . 27

7.6.5. Priority Arbitration . . . 27
7.7. Magic Device (Simulation Only) . . . 28
7.7.1. Overview . . . 28
7.7.2. Register Map . . . 28
7.7.3. Exit Code Encoding . . . 28
8. AXI Interconnect and Bus Protocol . . . 30
8.1. AXI4-Lite Overview . . . 30
8.2. Crossbar Architecture . . . 30
8.2.1. Address Decode Logic . . . 30
8.2.2. Error Responses . . . 31
8.3. AXI4 Memory Interface . . . 31
8.3.1. I-Cache Burst Transactions . . . 31
8.3.2. Data Access Transactions . . . 32
8.4. Bus Performance . . . 32
9. Electrical and Timing Characteristics . . . 33
9.1. Clock and Reset . . . 33
9.2. Timing Parameters . . . 33
9.3. Power Consumption (Estimated) . . . 33
9.4. Environmental Specifications . . . 34
10. Programming Guide . . . 35
10.1. Boot Sequence . . . 35
10.2. Memory Initialization . . . 35
10.3. Interrupt Handling . . . 35
10.3.1. Setting Up Interrupts . . . 35
10.3.2. Trap Handler Example . . . 36
10.4. Peripheral Usage Examples . . . 37
10.4.1. UART Echo Program . . . 37
10.4.2. SPI Flash Read . . . 37
10.4.3. I2C EEPROM Write . . . 38
10.4.4. DMA Memory Copy . . . 38
10.5. Performance Optimization Tips . . . 39
10.6. Debugging . . . 39
10.6.1. Simulation Debug . . . 39
10.6.2. Waveform Analysis . . . 39
10.6.3. Instruction Trace Comparison . . . 40
11. Conclusion . . . 41
Appendix A: Appendix A: Register Quick Reference . . . 42
A.1. CLINT Registers . . . 42
A.2. PLIC Registers . . . 42
A.3. UART Registers . . . 42
A.4. SPI Registers . . . 42

A.5. I2C Registers . . . 43
A.6. DMA Global Registers . . . 43
Appendix B: Appendix B: Interrupt Vector Table . . . 44
Appendix C: Appendix C: Pin Descriptions . . . 45
C.1. Clock and Reset . . . 45
C.2. UART Interface . . . 45
C.3. SPI Interface . . . 45
C.4. I2C Interface . . . 45
C.5. External AXI4 Memory Interface . . . 45
Appendix D: Appendix D: Revision History . . . 47

Chapter 1. Abstract

This document provides comprehensive technical specifications for the RV32IMA System-on-Chip (SoC), a high-performance 32-bit RISC-V processor platform. The SoC features a 5-stage pipelined RV32IMA core with instruction cache, AXI4/AXI4-Lite interconnect, and a rich set of peripherals including UART, SPI, I2C, DMA, timer, and interrupt controller. Designed for embedded applications, the SoC supports both RTL simulation and FPGA implementation.

Chapter 2. Introduction

2.1. Overview

The RV32IMA SoC is a complete embedded processor system implementing the RISC-V RV32IMA instruction set architecture with Machine mode (M-mode) privilege support. The system integrates:

- **RV32IMA Processor Core**: 5-stage pipeline with instruction cache, store buffer, and instruction buffer
- **Memory Subsystem**: 2MB external RAM with AXI4 interface, configurable instruction cache
- **Interconnect**: AXI4-Lite crossbar interconnect supporting multiple master and slave devices
- **Standard Peripherals**: UART, SPI, I2C, DMA with FIFO-buffered interfaces
- **Interrupt System**: CLINT (Core-Local Interrupt) for timer/software interrupts, PLIC (Platform-Level Interrupt Controller) for external peripherals
- **Debug Support**: Magic device for simulation control and console I/O

2.2. Key Features

Feature	Specification
ISA	RV32IMA_Zicsr (Integer, Multiply/Divide, Atomic, CSR)
Pipeline	5-stage in-order (Fetch, Decode, Execute, Memory, Writeback)
Instruction Cache	Configurable size (default 4KB), associativity (default 2-way set-associative), line size (default 32B)
Privilege Modes	Machine (M-mode) only
Bus Protocol	AXI4-Lite (peripherals), AXI4 (memory)
Clock Frequency	100 MHz (default, parameterizable)
Memory	2MB external RAM, supports up to 512MB address space
Peripherals	UART, SPI, I2C, DMA (up to 8 channels), CLINT, PLIC, Magic device
Interrupts	Timer, software, external (PLIC-routed)
Performance	~2.3 CPI typical, up to ~99% I-cache hit rate (workload dependent)
Implementation	RTL simulation (Verilator), FPGA synthesis supported

2.3. Document Organization

This datasheet is organized as follows:

- **Section 2**: System architecture overview with block diagram
- **Section 3**: Memory map and address decode
- **Section 4**: Processor core specification

- **Section 5**: Instruction cache specification
- **Section 6**: Peripheral specifications (UART, SPI, I2C, DMA, CLINT, PLIC, Magic device)
- **Section 7**: AXI interconnect and bus protocol
- **Section 8**: Electrical and timing characteristics
- **Section 9**: Programming guide
- **Appendices**: Register quick reference, interrupt vectors, pin descriptions

Chapter 3. System Architecture

3.1. Block Diagram

```
                    +-----------------------------------------------------+
                    |           RV32IMA SoC Top Level                     |
                    |                                                     |
+-----------------+-----------------------------------------------+     |
|  RV32 Core      |                                               |     |
|  +------------+ |                                               |     |
|  |   IF       |-+->Instruction                                  |     |
|  |   (Fetch)  |<+--  Buffer                                     |     |
|  +-----+------+ |   (IB_DEPTH)                                    |     |
|        |        |        |                                        |     |
|  +-----v------+ |        v                                        |     |
|  |   ID       | |   +----------+         +-------------+          |     |
|  |  (Decode)  | |   | I-Cache  |<------->|  Mem Arbiter|          |     |
|  +-----+------+ |   | (4KB/2W) |         |  (I/D paths)|          |     |
|        |        |   +----------+         +------+------+          |     |
|  +-----v------+ |                               |                 |     |
|  |   EX       | |                               v                 |     |
|  | (Execute)  | |                        +-------------+          |     |
|  +-----+------+ |                        | AXI4-Lite   |          |     |
|        |        |                        |  Crossbar   |          |     |
|  +-----v------+ |                        |  (1-to-8)   |          |     |
|  |   MEM      |-+->Store                 +------+------+          |     |
|  |  (Memory)  |<+--Buffer                       |                 |     |
|  +-----+------+ |  (SB_DEPTH)                   |                 |     |
|        |        |                               |                 |     |
|  +-----v------+ |   +---------------------------+-----------------+---+
|  |   WB       | |   | Slave 0: RAM (2MB)        |                 |     |
|  | (Writeback)| |   | 0x8000_0000-0x801F_FFFF   v                 |     |
|  +------------+ |   |                    +-------------+          |     |
|                 |   |                    |  AXI4 RAM   |          |     |
+-----------------+   |                    |  (2MB ext)  |          |     |
                      |                    +-------------+          |     |
                      |                                             |     |
                      | Slave 1: CLINT    0x0200_0000-0x020B_FFFF   |     |
                      |   +--------------------------+              |     |
                      |   | Timer (mtime/mtimecmp)   |              |     |
                      |   | Software IRQ (msip)      |---->timer_irq|     |
                      |   +--------------------------+   +>sw_irq   |     |
                      |                                             |     |
                      | Slave 2: PLIC     0x0C00_0000-0x0FFF_FFFF   |     |
                      |   +--------------------------+              |     |
                      |   | Priority/Enable/Claim    |<---uart_irq  |     |
                      |   | Threshold/Complete       |<---spi_irq   |     |
                      |   +--------------+-----------+<---i2c_irq   |     |
                      |                  |            <---dma_irq   |     |
```

```
                        |                  +-------------->ext_irq    |   |
                        |                                             |   |
                        | Slave 3: UART     0x2000_0000-0x2000_FFFF   |   |
                        |   +-------------------------+              |   |
                        |   | TX/RX FIFOs (16-deep)    |<---uart_rx   |   |
                        |   | Baud rate: 25MHz max     |--->uart_tx   |   |
                        |   +-------------------------+              |   |
                        |                                             |   |
                        | Slave 4: I2C      0x2001_0000-0x2001_FFFF   |   |
                        |   +-------------------------+              |   |
                        |   | Master mode (100k/400k)  |<-->i2c_sda   |   |
                        |   | TX/RX FIFOs (8-deep)     |<-->i2c_scl   |   |
                        |   +-------------------------+              |   |
                        |                                             |   |
                        | Slave 5: SPI      0x2002_0000-0x2002_FFFF   |   |
                        |   +-------------------------+              |   |
                        |   | Master mode (4 CS lines) |--->spi_sclk  |   |
                        |   | TX/RX FIFOs (8-deep)     |--->spi_mosi  |   |
                        |   | Modes 0-3 (CPOL/CPHA)    |<---spi_miso  |   |
                        |   +-------------------------+--->spi_cs_n  |   |
                        |                                             |   |
                        | Slave 6: DMA      0x2003_0000-0x2003_0FFF   |   |
                        |   +-------------------------+              |   |
                        |   | 8 channels, 1D/2D/3D/SG  |<-->(AXI4     |   |
                        |   | 32-bit data width        |    master    |   |
                        |   | Performance counters     |    port)     |   |
                        |   +-------------------------+              |   |
                        |                                             |   |
                        | Slave 7: Magic    0xFFFF_0000-0xFFFF_FFFF   |   |
                        |   +-------------------------+              |   |
                        |   | Simulation console I/O   |              |   |
                        |   | Exit code control        |              |   |
                        |   +-------------------------+              |   |
                        +---------------------------------------------+   |
                                                                          |
    External Pins:                                                        |
      clk, rst_n                                                          |
      uart_tx, uart_rx                                                    |
      spi_sclk, spi_mosi, spi_miso, spi_cs_n[3:0]                         |
      i2c_scl_o, i2c_scl_i, i2c_scl_oe                                    |
      i2c_sda_o, i2c_sda_i, i2c_sda_oe                                    |
      m_axi_* (external RAM interface)                                    |
                                                                          |
```

3.2. System Components

3.2.1. Processor Core (rv32_core)

The RV32IMA processor core implements a 5-stage pipeline with the following key features:

- **Instruction Fetch (IF)**: Fetches instructions from I-cache or memory via instruction buffer
- **Instruction Decode (ID)**: Decodes instructions, reads register file
- **Execute (EX)**: Executes ALU operations, computes addresses, evaluates branches
- **Memory (MEM)**: Accesses data memory via store buffer
- **Writeback (WB)**: Writes results to register file, retires instructions

The core supports:

- Data forwarding to resolve RAW hazards
- Pipeline stalls for load-use dependencies
- Branch prediction (static not-taken)
- Configurable multiply/divide (fast combinatorial or slow serial)
- Instruction buffer (IB_DEPTH=4) for outstanding instruction fetches
- Store buffer (SB_DEPTH=2) for non-blocking stores

3.2.2. Instruction Cache (rv32_icache)

An optional configurable instruction cache improves performance by reducing memory latency:

- **Size**: 4KB default (parameterizable)
- **Associativity**: 2-way set-associative (parameterizable)
- **Line Size**: 32 bytes (8 words)
- **Replacement Policy**: Round-robin per set
- **Bypass Mode**: Cacheable or direct-mapped via CMO operations
- **AXI Interface**: WRAP bursts for cache-line fills (critical-word-first)

3.2.3. Memory Subsystem

- **External RAM**: 2MB AXI4 SRAM at 0x8000_0000, supports burst reads/writes
- **Instruction Arbiter**: Arbitrates between instruction fetch and data memory paths
- **AXI Crossbar**: 1-to-8 interconnect routes transactions to appropriate slaves

3.2.4. Interrupt System

- **CLINT**: Machine-mode timer (mtime/mtimecmp) and software interrupts (msip)
- **PLIC**: Routes up to 31 external interrupt sources with programmable priorities
- **Core Integration**: timer_irq, software_irq, external_irq inputs to core CSR unit

Chapter 4. Memory Map

4.1. Address Space Overview

The RV32IMA SoC implements a 32-bit address space (4GB) with the following allocation:

Table 1. Complete Memory Map

Address Range	Device	Size	Description
0x0000_0000 - 0x0001_FFFF	ROM (reserved)	128KB	Reserved for future boot ROM
0x0200_0000 - 0x020B_FFFF	CLINT	768KB	(only 64KB used, rest reserved per RISC-V spec)
0x0C00_0000 - 0x0FFF_FFFF	PLIC	64MB	(actual registers much smaller, per RISC-V spec)
0x2000_0000 - 0x2000_FFFF	UART	64KB	Serial communication peripheral
0x2001_0000 - 0x2001_FFFF	I2C	64KB	I2C master controller
0x2002_0000 - 0x2002_FFFF	SPI	64KB	SPI master controller
0x2003_0000 - 0x2003_0FFF	DMA	4KB	DMA controller (8 channels)
0x8000_0000 - 0x801F_FFFF	RAM	2MB	Main system memory (SRAM)
0xFFFF_0000 - 0xFFFF_FFFF	Magic Device	64KB	Simulation control and console

4.2. Address Decode Rules

The AXI crossbar (`axi_xbar`) decodes addresses based on the high-order bits:

- **Slave 0 (RAM)**: `addr[31:21] == 11'h400` (0x8000_0000)
- **Slave 1 (CLINT)**: `addr[31:20] == 12'h020` (0x0200_0000)
- **Slave 2 (PLIC)**: `addr[31:26] == 6'h03` (0x0C00_0000)
- **Slave 3 (UART)**: `addr[31:16] == 16'h2000` (0x2000_0000)
- **Slave 4 (I2C)**: `addr[31:16] == 16'h2001` (0x2001_0000)
- **Slave 5 (SPI)**: `addr[31:16] == 16'h2002` (0x2002_0000)
- **Slave 6 (DMA)**: `addr[31:12] == 20'h2003_0` (0x2003_0000)
- **Slave 7 (Magic)**: `addr[31:16] == 16'hFFFF` (0xFFFF_0000)

Unmapped addresses return an AXI SLVERR (2’b10) response on the read channel and OKAY (2’b00) on the write channel (write is silently ignored).

Chapter 5. Processor Core Specification

5.1. RV32IMA ISA Support

The processor core implements the following RISC-V instruction set extensions:

Table 2. Supported ISA Extensions

Extension	Description
RV32I	Base integer instruction set (32-bit)
M	Integer multiplication and division
A	Atomic instructions (LR/SC, AMO)
Zicsr	CSR access instructions (CSRRW, CSRRS, CSRRC, etc.)
Zifencei	Instruction-fetch fence (FENCE.I)

The core supports Machine mode (M-mode) only. User mode (U-mode) and Supervisor mode (S-mode) are not implemented.

5.2. Pipeline Architecture

The core uses a 5-stage in-order pipeline:

Table 3. Pipeline Stages

Stage	Name	Function
IF	Instruction Fetch	Request instruction from I-cache/memory via instruction buffer
ID	Instruction Decode	Decode opcode, read source registers, generate control signals
EX	Execute	Execute ALU operation, compute memory address, evaluate branch condition
MEM	Memory Access	Perform load/store via data memory interface and store buffer
WB	Writeback	Write result to destination register, retire instruction

5.2.1. Instruction Buffer (IB)

The instruction buffer decouples instruction fetch from pipeline execution:

- **Depth**: IB_DEPTH (default 4, parameterizable)
- **Purpose**: Allow multiple outstanding instruction fetches to hide memory latency
- **Benefit**: Keeps pipeline fed even during I-cache misses

5.2.2. Store Buffer (SB)

The store buffer allows stores to complete asynchronously:

- **Depth**: SB_DEPTH (default 2, parameterizable)
- **Purpose**: Buffer store requests so pipeline doesn't stall on store completion
- **Ordering**: Maintains program order for stores

5.2.3. Forwarding and Hazards

- **Data Forwarding**: EX→EX, MEM→EX, WB→EX paths resolve RAW hazards
- **Load-Use Stall**: Pipeline stalls 1 cycle when load result is immediately used
- **Branch Prediction**: Static not-taken; 2-cycle penalty on taken branches (flush IF and ID stages)
- **Structural Hazards**: Single memory port shared between I-fetch and D-access via arbiter

5.3. Performance Characteristics

Typical performance metrics (workload-dependent):

Table 4. Performance Metrics

Metric	Typical Value
CPI (Cycles Per Instruction)	~2.3 (with I-cache enabled, ~99% hit rate)
I-Cache Hit Rate	~99% (depends on working set size)
Branch Misprediction Penalty	2 cycles (flush IF and ID stages)
Load-Use Penalty	1 cycle stall
Multiply Latency	1 cycle (FAST_MUL=1), 32 cycles (FAST_MUL=0)
Divide Latency	1 cycle (FAST_DIV=1), 33 cycles (FAST_DIV=0)

5.4. CSR (Control and Status Registers)

The core implements Machine-mode CSRs per RISC-V Privileged Spec v1.11:

Table 5. Implemented CSRs

Address	Name	Access	Description
0x300	mstatus	R/W	Machine status register
0x301	misa	R/W	ISA and extensions (reports RV32IMA)
0x304	mie	R/W	Machine interrupt-enable register
0x305	mtvec	R/W	Machine trap-handler base address
0x340	mscratch	R/W	Machine scratch register

Address	Name	Access	Description
0x341	mepc	R/W	Machine exception program counter
0x342	mcause	R/W	Machine trap cause
0x343	mtval	R/W	Machine trap value
0x344	mip	R/W	Machine interrupt-pending register
0xB00	mcycle	R/W	Machine cycle counter (lower 32 bits)
0xB80	mcycleh	R/W	Machine cycle counter (upper 32 bits)
0xB02	minstret	R/W	Machine instructions-retired counter (lower 32 bits)
0xB82	minstreth	R/W	Machine instructions-retired counter (upper 32 bits)
0xF11	mvendorid	R/O	Vendor ID (returns 0)
0xF12	marchid	R/O	Architecture ID (returns 0)
0xF13	mimpid	R/O	Implementation ID (returns 0)
0xF14	mhartid	R/O	Hardware thread ID (returns 0)

5.5. Exception and Interrupt Handling

5.5.1. Exception Types

Table 6. Exception Causes (mcause[31]=0)

Code	Name	Description
0	Instruction address misaligned	PC not 4-byte aligned (should not occur in normal operation)
1	Instruction access fault	I-fetch AXI error response
2	Illegal instruction	Undefined opcode or malformed instruction
3	Breakpoint	EBREAK instruction executed
4	Load address misaligned	Load address not naturally aligned
5	Load access fault	Load AXI error response
6	Store/AMO address misaligned	Store address not naturally aligned
7	Store/AMO access fault	Store AXI error response
11	Environment call from M-mode	ECALL instruction executed in M-mode

5.5.2. Interrupt Types

Table 7. Interrupt Causes (mcause[31]=1)

Code	Name	Source
3	Machine software interrupt	CLINT MSIP register
7	Machine timer interrupt	CLINT mtime >= mtimecmp
11	Machine external interrupt	PLIC aggregated external interrupt

5.5.3. Trap Handling

On exception or interrupt:

1. PC is saved to `mepc`
2. Trap cause is written to `mcause`
3. Trap value (e.g., faulting address) is written to `mtval`
4. `mstatus.MIE` is cleared (disable interrupts)
5. PC is set to `mtvec` (trap vector address)

Return from trap: `MRET` instruction restores `mepc` to PC and restores interrupt enable.

Chapter 6. Instruction Cache Specification

6.1. Overview

The instruction cache (rv32_icache) reduces instruction fetch latency by caching frequently accessed code:

Table 8. I-Cache Configuration

Parameter	Default Value
CACHE_SIZE	4096 bytes (4KB)
CACHE_LINE_SIZE	32 bytes (8 words)
CACHE_WAYS	2 (2-way set-associative)
Number of Sets	64 (computed: SIZE / (LINE_SIZE × WAYS))
Replacement Policy	Round-robin (per-set victim pointer)
Write Policy	N/A (instruction cache is read-only)

6.2. Cache Organization

- **Address Decomposition**:
  - Tag: bits [31:11] (21 bits)
  - Index: bits [10:5] (6 bits, selects 1 of 64 sets)
  - Word Offset: bits [4:2] (3 bits, selects 1 of 8 words)
  - Byte Offset: bits [1:0] (always 00 for aligned word access)
- **Storage**:
  - Valid bits: flip-flops (2 ways × 64 sets = 128 bits)
  - Tag storage: SRAM macros (2 instances, 64 entries × 21 bits)
  - Data storage: SRAM macros (2 instances, 512 entries × 32 bits)

6.3. Cache Operations

6.3.1. Normal Fetch (Cache Hit)

1. Core asserts `imem_req_valid` with address
2. Cache performs lookup (1 cycle)
3. On hit, returns data via `imem_resp_data` next cycle
4. Core accepts response and continues

Latency: 1 cycle (registered output)

6.3.2. Cache Miss

1. Cache detects miss in lookup stage
2. Issues AXI WRAP burst for entire cache line (8 beats)
3. Returns critical word first (early restart) to unblock core
4. Remaining beats drain in background while core continues
5. Victim way is selected via round-robin pointer

Latency: Memory latency + 1 cycle (critical-word-first optimization)

6.3.3. Bypass Mode

When cache is disabled via CMO (Cache Management Operation):

- Every fetch issues single-beat AXI INCR transaction
- No caching occurs, data is forwarded directly to core
- Useful for debugging or non-cacheable code regions

6.4. Cache Management Operations (CMO)

The cache supports software-initiated management via the `cmo_*` sideband interface:

Table 9. Supported CMO Operations

Opcode	Name	Description
2'b00	CMO_INVAL	Invalidate specific cache line matching address
2'b01	CMO_DISABLE	Disable cache, enter bypass mode
2'b10	CMO_ENABLE	Re-enable cache
2'b11	CMO_FLUSH_ALL	Invalidate all cache entries (FENCE.I support)

CMO operations are typically triggered by `FENCE.I` or `cbo.inval` instructions in the core's MEM stage.

6.5. Performance Counters (Simulation Only)

The cache provides performance monitoring signals (not synthesized):

- `perf_req_cnt`: Total fetch requests
- `perf_hit_cnt`: Cache hits
- `perf_miss_cnt`: Cache misses
- `perf_bypass_cnt`: Bypass fetches (cache disabled)
- `perf_fill_cnt`: Completed line fills
- `perf_cmo_cnt`: CMO operations executed

Chapter 7. Peripheral Specifications

7.1. UART (Universal Asynchronous Receiver/Transmitter)

7.1.1. Overview

The UART peripheral provides serial communication with configurable baud rate and 8N1 format (8 data bits, no parity, 1 stop bit).

Table 10. UART Features

Feature	Specification
Base Address	0x2000_0000
Size	64KB (only first 0x18 bytes used)
Data Format	8 bits, no parity, 1 stop bit (8N1)
Baud Rate	Configurable (default 25 Mbaud, min 4 clocks/bit)
TX FIFO Depth	16 entries
RX FIFO Depth	16 entries
Interrupt	PLIC source ID 1

7.1.2. Register Map

Table 11. UART Registers

Offset	Name	Access	Description
0x00	DATA	R/W	**Read**: Pop byte from RX FIFO **Write**: Push byte to TX FIFO
0x04	STATUS	R/O	**[0]**: tx_busy (1=TX FIFO full, can't write) **[1]**: tx_full (same as bit 0) **[2]**: rx_ready (1=RX FIFO not empty, can read) **[3]**: rx_full (1=RX FIFO full)
0x08	IE (Interrupt Enable)	R/W	**[0]**: Enable RX not-empty interrupt **[1]**: Enable TX empty interrupt
0x0C	IS (Interrupt Status)	R/O	**[0]**: RX not-empty (level) **[1]**: TX empty (level)
0x10	LEVEL	R/O	**[3:0]**: RX FIFO count **[11:8]**: TX FIFO count
0x14	CTRL	R/W	**[0]**: Loopback enable (1=TX→RX internal loopback)

Interrupt asserts when `(IE & IS) != 0`.

7.1.3. Baud Rate Configuration

The UART baud rate is determined by the SoC parameter `BAUD_RATE`:

```
CLKS_PER_BIT = CLK_FREQ / BAUD_RATE
```

Hardware Constraint: `CLKS_PER_BIT >= 4` (absolute minimum for RX sampling).

Recommended: `CLKS_PER_BIT >= 8` for production use (provides timing margin).

Table 12. Common Baud Rate Examples (100 MHz Clock)

Baud Rate	CLKS_PER_BIT	Notes
115200	868	Standard slow baud, large margin
921600	109	Fast baud, good margin
1 Mbaud	100	Round number, good margin
3.125 Mbaud	32	Fast, acceptable margin
12.5 Mbaud	8	Very fast, minimum recommended
25 Mbaud	4	Maximum (hardware minimum, lab use only)

Baud rates requiring `CLKS_PER_BIT < 4` will not work correctly due to insufficient sampling resolution in the RX state machine.

7.1.4. Interrupt Behavior

- **RX Interrupt**: Fires when RX FIFO transitions from empty to non-empty
- **TX Interrupt**: Fires when TX FIFO becomes empty (all data transmitted)

Software should:

- **RX ISR**: Read all available bytes from DATA register until STATUS[2]=0
- **TX ISR**: Refill TX FIFO by writing to DATA register

7.1.5. External Pins

Table 13. UART Pins

Signal	Direction	Description
uart_tx	Output	UART transmit data (to external RX pin)
uart_rx	Input	UART receive data (from external TX pin)

Both signals are asynchronous to system clock (standard UART timing).

7.2. SPI (Serial Peripheral Interface)

7.2.1. Overview

The SPI peripheral provides master-mode serial communication for interfacing with SPI slaves (flash, ADC, sensors, etc.).

Table 14. SPI Features

Feature	Specification
Base Address	0x2002_0000
Size	64KB (only first 0x1C bytes used)
Mode	Master only
Clock Modes	Modes 0-3 (CPOL/CPHA configurable)
Data Width	8 bits per transfer
Clock Frequency	Programmable divider: `SCLK = CLK / (2 * (DIV + 1))`
TX FIFO Depth	8 entries
RX FIFO Depth	8 entries
Chip Selects	4 (active-low)
Interrupt	PLIC source ID 2

7.2.2. Register Map

Table 15. SPI Registers

Offset	Name	Access	Description
0x00	CTRL	R/W	**[0]**: Enable (1=enable SPI controller) **[1]**: CPOL (clock polarity: 0=idle low, 1=idle high) **[2]**: CPHA (clock phase: 0=leading edge, 1=trailing edge) **[3]**: Loopback (1=MOSI→MISO internal loopback) **[7:4]**: CS chip select (active-low, 4 bits)
0x04	DIV	R/W	Clock divider: `SCLK = CLK / (2 * (DIV + 1))` DIV=0 → CLK/2 (fastest), DIV=255 → CLK/512
0x08	TX (TX Data)	W/O	Write: push byte to TX FIFO (triggers transfer) Read: returns 0
0x0C	RX (RX Data)	R/O	Read: pop byte from RX FIFO Write: ignored

Offset	Name	Access	Description
0x10	STATUS	R/O	**[0]**: Busy (1=transfer in progress) **[1]**: TX ready (!tx_full, can accept data) **[2]**: RX valid (!rx_empty, data available) **[3]**: TX empty (TX FIFO drained) **[4]**: RX full (RX FIFO full)
0x14	IE (Interrupt Enable)	R/W	**[0]**: Enable RX not-empty interrupt **[1]**: Enable TX empty interrupt
0x18	IS (Interrupt Status)	R/O	**[0]**: RX not-empty (level) **[1]**: TX empty (level)

7.2.3. SPI Modes

Table 16. SPI Clock Modes

Mode	CPOL	CPHA	Description
0	0	0	Clock idle low, sample on leading (rising) edge
1	0	1	Clock idle low, sample on trailing (falling) edge
2	1	0	Clock idle high, sample on leading (falling) edge
3	1	1	Clock idle high, sample on trailing (rising) edge

7.2.4. Clock Divider Examples (100 MHz System Clock)

Table 17. Common SPI Clock Frequencies

DIV Value	SCLK Frequency	Application
0	50 MHz	Maximum speed (may exceed slave limits)
1	33.33 MHz	Very fast SPI (high-speed flash)
3	25 MHz	Fast SPI (typical maximum for many devices)
9	10 MHz	Standard SPI
24	4 MHz	Conservative SPI
49	2 MHz	Slow devices (SD card init)
99	1 MHz	Very slow devices

7.2.5. Usage Flow

1. Configure CTRL[7:4] to select chip (active-low: clear bit to select)
2. Set CTRL[2:1] for desired CPOL/CPHA mode
3. Set DIV for desired SCLK frequency
4. Set CTRL[0]=1 to enable controller

5. Write data to TX register to initiate transfer
6. Poll STATUS[2] or wait for RX interrupt
7. Read received data from RX register
8. Repeat steps 5-7 for each byte
9. Set CTRL[7:4]=0xF to deselect all chips (all CS high)

7.2.6. External Pins

Table 18. SPI Pins

Signal	Direction	Description
spi_sclk	Output	SPI serial clock
spi_mosi	Output	Master Out, Slave In (data to slave)
spi_miso	Input	Master In, Slave Out (data from slave)
spi_cs_n[3:0]	Output	Chip select lines (active-low)

7.3. I2C (Inter-Integrated Circuit)

7.3.1. Overview

The I2C peripheral provides master-mode I2C communication for interfacing with sensors, EEPROMs, and other I2C devices.

Table 19. I2C Features

Feature	Specification
Base Address	0x2001_0000
Size	64KB (only first 0x1C bytes used)
Mode	Master only
Speed	Standard (100 kHz) and Fast (400 kHz)
Addressing	7-bit address mode
TX FIFO Depth	8 entries
RX FIFO Depth	8 entries
Interrupt	PLIC source ID 3

7.3.2. Register Map

Table 20. I2C Registers

Offset	Name	Access	Description
0x00	CTRL	R/W	**[0]**: Enable (1=enable I2C controller) **[1]**: Start (write 1 to issue START condition) **[2]**: Stop (write 1 to issue STOP condition) **[3]**: Read (1=read byte from slave, 0=write byte) **[4]**: NACK (1=send NACK after read, 0=send ACK)
0x04	DIV	R/W	Clock divider: `SCL_period = CLK / (4 * (DIV + 1))` For 100kHz: DIV = (CLK_FREQ / 400000) - 1 For 400kHz: DIV = (CLK_FREQ / 1600000) - 1
0x08	TX (TX Data)	W/O	Write: push byte to TX FIFO Read: returns 0
0x0C	RX (RX Data)	R/O	Read: pop byte from RX FIFO Write: ignored
0x10	STATUS	R/O	**[0]**: Busy (1=transfer in progress) **[1]**: TX ready (!tx_full, can accept data) **[2]**: RX valid (!rx_empty, data available) **[3]**: ACK received (1=slave ACKed last byte)
0x14	IE (Interrupt Enable)	R/W	**[0]**: Enable RX not-empty interrupt **[1]**: Enable TX empty interrupt **[2]**: Enable STOP done interrupt
0x18	IS (Interrupt Status)	R/O	**[0]**: RX not-empty (level) **[1]**: TX empty (level) **[2]**: STOP done (level)

7.3.3. Clock Divider Examples (100 MHz System Clock)

Table 21. Common I2C Speeds

Speed	DIV Value	Formula
100 kHz (Standard)	249	(100M / 400k) - 1
400 kHz (Fast)	62	(100M / 1.6M) - 1
1 MHz (Fast-mode Plus)	24	(100M / 4M) - 1

The I2C bus frequency formula accounts for 4 clock cycles per SCL period (rise/fall/hold times in the state machine).

7.3.4. Usage Flow

Write Transaction

1. Write slave address (shifted left, bit 0=0 for write) to TX register
2. Set CTRL[1]=1 to issue START
3. Wait for STATUS[1]=1 (TX ready) or TX interrupt

4. Write data bytes to TX register
5. Check STATUS[3]=1 to verify slave ACK
6. Set CTRL[2]=1 to issue STOP
7. Wait for IS[2]=1 (STOP done interrupt)

Read Transaction

1. Write slave address (shifted left, bit 0=1 for read) to TX register
2. Set CTRL[1]=1 to issue START
3. Set CTRL[3]=1 (read mode)
4. For each byte:
    - Wait for STATUS[2]=1 or RX interrupt
    - Read byte from RX register
    - For last byte, set CTRL[4]=1 (NACK)
5. Set CTRL[2]=1 to issue STOP

7.3.5. External Pins

Table 22. I2C Pins

Signal	Direction	Description
i2c_scl_o	Output	SCL output data (0=pull low)
i2c_scl_i	Input	SCL input data (for clock stretching)
i2c_scl_oe	Output	SCL output enable (1=high-Z, 0=drive)
i2c_sda_o	Output	SDA output data (0=pull low)
i2c_sda_i	Input	SDA input data
i2c_sda_oe	Output	SDA output enable (1=high-Z, 0=drive)

I2C uses open-drain signaling. External pull-up resistors (typically 4.7kΩ) are required on both SCL and SDA lines.

7.4. DMA (Direct Memory Access Controller)

7.4.1. Overview

The DMA controller provides high-performance memory-to-memory and peripheral-to-memory transfers with minimal CPU intervention.

Table 23. DMA Features

Feature	Specification
Base Address	0x2003_0000
Size	4KB
Channels	8 independent channels
Transfer Modes	1D (linear), 2D (strided), 3D (planar), Scatter-Gather
Data Width	32 bits (parameterizable to 64/128)
Max Burst Length	16 beats (parameterizable)
FIFO Depth	16 entries per channel (parameterizable)
Interrupt	PLIC source ID 4 (aggregated, all channels)

7.4.2. Register Map

Each channel has registers at offset `BASE + channel * 0x40`:

*Table 24. Per-Channel Registers (offset from BASE + ch*0x40)*

Offset	Name	Access	Description
0x00	CTRL	R/W	**[0]**: EN (enable channel) **[1]**: START (write 1 to arm transfer, auto-clears) **[2]**: STOP (write 1 to abort, auto-clears) **[4:3]**: MODE (00=1D, 01=2D, 10=3D, 11=SG) **[5]**: SRC_INC (1=increment source address) **[6]**: DST_INC (1=increment destination address) **[7]**: IE (interrupt enable for this channel)
0x04	STAT	R/W1C	**[0]**: BUSY (transfer in progress, read-only) **[1]**: DONE (transfer complete, write 1 to clear) **[2]**: ERR (AXI error, write 1 to clear)
0x08	SRC_ADDR	R/W	Source base address (must be 4-byte aligned)
0x0C	DST_ADDR	R/W	Destination base address (must be 4-byte aligned)
0x10	XFER_CNT	R/W	Transfer size in bytes (1D: total; 2D/3D: bytes per row)
0x14	SRC_STRIDE	R/W	Source stride in bytes between rows (2D/3D only)
0x18	DST_STRIDE	R/W	Destination stride in bytes between rows (2D/3D only)
0x1C	ROW_CNT	R/W	Number of rows (2D/3D only)
0x20	SRC_PLANE_STRIDE	R/W	Source stride in bytes between planes (3D only)
0x24	DST_PLANE_STRIDE	R/W	Destination stride in bytes between planes (3D only)
0x28	PLANE_CNT	R/W	Number of planes (3D only)

Offset	Name	Access	Description
0x2C	SG_ADDR	R/W	Scatter-gather descriptor list base address (SG mode)
0x30	SG_CNT	R/W	Number of scatter-gather descriptors (SG mode)

Table 25. Global Registers (offset from BASE)

Offset	Name	Access	Description
0xF00	IRQ_STAT	R/W1C	Channel interrupt status (bit N = channel N done/err, write 1 to clear)
0xF04	IRQ_EN	R/W	Per-channel IRQ global enable (bit N = channel N)
0xF08	DMA_ID	R/O	DMA controller ID register (reads 0xD4A0_0100)
0xF10	PERF_CTRL	R/W	**[0]**: Enable performance counters **[1]**: Write 1 to reset all counters
0xF14	PERF_CYCLES	R/O	Total cycles elapsed while PERF_CTRL[0]=1
0xF18	PERF_RD_BYTES	R/O	Total bytes read via AXI (cumulative)
0xF1C	PERF_WR_BYTES	R/O	Total bytes written via AXI (cumulative)

7.4.3. Transfer Modes

1D Transfer (Linear)

Simple contiguous memory copy:

- Set `SRC_ADDR`, `DST_ADDR`, `XFER_CNT`
- Set `CTRL[4:3]=00` (MODE=1D)
- Set `CTRL[5:6]` for address increment behavior
- Set `CTRL[1]=1` to start

2D Transfer (Strided)

Copy 2D arrays with row stride:

- Set `SRC_ADDR`, `DST_ADDR`, `XFER_CNT` (bytes per row)
- Set `SRC_STRIDE`, `DST_STRIDE` (bytes between row starts)
- Set `ROW_CNT` (number of rows)
- Set `CTRL[4:3]=01` (MODE=2D)

Example: Copy 16×16 pixel subregion from 640×480 framebuffer:

```
XFER_CNT = 16 * 4        // 16 pixels x 4 bytes/pixel
SRC_STRIDE = 640 * 4     // Source pitch
```

```
DST_STRIDE = 16 * 4      // Destination pitch (packed)
ROW_CNT = 16             // 16 rows
```

3D Transfer (Planar)

Copy 3D volumes with plane stride:

- Configure as 2D, plus set `SRC_PLANE_STRIDE`, `DST_PLANE_STRIDE`, `PLANE_CNT`
- Set `CTRL[4:3]=10` (MODE=3D)

Scatter-Gather Transfer

Chain multiple transfers from descriptor list:

- Each descriptor is 16 bytes: `{src_addr, dst_addr, xfer_cnt, mode_ctrl}`
- Set `SG_ADDR` to descriptor list base address
- Set `SG_CNT` to number of descriptors
- Set `CTRL[4:3]=11` (MODE=SG)

7.4.4. Interrupt Behavior

- Each channel can raise interrupt on completion (DONE) or error (ERR)
- Channel interrupts are aggregated to single PLIC source (ID 4)
- Software reads `IRQ_STAT` to determine which channel(s) triggered
- Write 1 to `IRQ_STAT` bits to acknowledge

7.4.5. Performance Monitoring

Enable performance counters to measure DMA throughput:

```
DMA_PERF_CTRL = 0x3;    // Enable counters and reset
// ... run workload ...
cycles = DMA_PERF_CYCLES;
rd_bytes = DMA_PERF_RD_BYTES;
wr_bytes = DMA_PERF_WR_BYTES;
bandwidth_MBps = (rd_bytes + wr_bytes) / cycles * CLK_FREQ / 1e6;
```

7.4.6. AXI Interface

The DMA controller has a dedicated AXI4 master port (32-bit data width default):

- Burst optimization: DMA controller generates INCR bursts up to `MAX_BURST_LEN` beats
- Page boundary handling: Bursts never cross 4KB AXI page boundaries
- Error handling: AXI SLVERR/DECERR sets channel STAT[2]=1 and aborts transfer

7.5. CLINT (Core-Local Interruptor)

7.5.1. Overview

The CLINT provides machine-mode timer and software interrupts per RISC-V Privileged Specification.

Table 26. CLINT Features

Feature	Specification
Base Address	0x0200_0000
Size	768KB (large per RISC-V spec, only 64KB used)
Functionality	64-bit real-time counter (mtime), 64-bit compare (mtimecmp), software interrupt (msip)
Timer Tick	1 per clock cycle (default), configurable in trace mode
Interrupt Outputs	timer_irq, software_irq to core

7.5.2. Register Map

Table 27. CLINT Registers

Offset	Name	Access	Description
0x0000	MSIP	R/W	Machine Software Interrupt Pending Write 1 to bit 0 to trigger software interrupt
0x4000	MTIMECMP_LO	R/W	Timer compare register (lower 32 bits)
0x4004	MTIMECMP_HI	R/W	Timer compare register (upper 32 bits)
0xBFF8	MTIME_LO	R/W	Real-time counter (lower 32 bits)
0xBFFC	MTIME_HI	R/W	Real-time counter (upper 32 bits)

Register offsets match RISC-V CLINT specification for single-hart systems.

7.5.3. Timer Operation

The 64-bit `mtime` counter increments by 1 every clock cycle. Timer interrupt fires when:

```
mtime >= mtimecmp
```

Software clears timer interrupt by writing a new (larger) value to `mtimecmp`.

Example: Set timer for 1ms from now (100 MHz clock):

```
uint64_t mtime = read_mtime();
uint64_t mtimecmp = mtime + 100000;  // 1ms = 100k cycles @ 100MHz
```

```
write_mtimecmp(mtimecmp);
```

7.5.4. Software Interrupt Operation

Write `MSIP[0] = 1` to trigger machine software interrupt immediately. Clear by writing `MSIP[0] = 0`.

Typical use: Inter-processor interrupt (IPI) on multi-hart systems, or self-interrupt for context switching.

7.6. PLIC (Platform-Level Interrupt Controller)

7.6.1. Overview

The PLIC routes external interrupt sources to the processor core with programmable priorities.

Table 28. PLIC Features

Feature	Specification
Base Address	0x0C00_0000
Size	64MB (large per RISC-V spec, only ~2MB used)
Interrupt Sources	Up to 31 (source 0 reserved)
Priority Levels	0-7 (0 = never interrupt, 1-7 = increasing priority)
Target Contexts	1 (hart 0, M-mode)
Claim/Complete Protocol	Standard RISC-V PLIC mechanism

7.6.2. Register Map

Table 29. PLIC Registers

Offset	Name	Access	Description
0x000000	PRIORITY[0]	R/O	Reserved, always reads 0
0x000004 × i	PRIORITY[i]	R/W	Priority for interrupt source i (i=1..31, 3-bit)
0x001000	PENDING	R/O	Interrupt pending bits [31:1] (bit i = source i pending)
0x002000	ENABLE	R/W	Interrupt enable bits [31:1] for context 0
0x200000	THRESHOLD	R/W	Priority threshold for context 0 (3-bit) Interrupts fire only if source priority > threshold

Offset	Name	Access	Description
0x200004	CLAIM/COMPLETE	R/W	**Read**: Claim highest-priority pending interrupt (returns source ID) **Write**: Complete interrupt (write source ID to clear)

7.6.3. Interrupt Sources

The following peripherals are connected to the PLIC:

Table 30. PLIC Interrupt Source IDs

Source ID	Peripheral	Description
1	UART	UART RX not-empty or TX empty
2	SPI	SPI RX not-empty or TX empty
3	I2C	I2C RX not-empty, TX empty, or STOP done
4	DMA	DMA channel done or error (aggregated)
5-31	Reserved	Future peripheral expansion

7.6.4. Interrupt Flow

1. **Pending**: Peripheral asserts `irq_src[i]`, PLIC sets `PENDING[i]=1`
2. **Arbitration**: PLIC selects highest-priority enabled pending interrupt where `priority[i] > threshold`
3. **Notification**: PLIC asserts `external_irq` to core
4. **Claim**: ISR reads `CLAIM` register to get source ID, `PENDING[i]` remains set
5. **Service**: ISR handles the interrupt, clears condition in peripheral
6. **Complete**: ISR writes source ID to `COMPLETE` register
7. **Clear**: If peripheral de-asserted `irq_src[i]`, PLIC clears `PENDING[i]`

> If peripheral IRQ remains asserted after complete, `PENDING[i]` is immediately re-set (level-sensitive behavior).

7.6.5. Priority Arbitration

When multiple interrupts are pending:

- Highest priority wins (priority 7 > priority 1)
- On priority tie, lowest source ID wins (source 1 > source 2)

Example configuration:

```
PRIORITY[1] = 5;  // UART high priority
```

```
PRIORITY[2] = 3;  // SPI medium priority
PRIORITY[3] = 3;  // I2C medium priority
PRIORITY[4] = 1;  // DMA low priority
THRESHOLD = 0;    // Allow all enabled interrupts
ENABLE = 0x1E;    // Enable sources 1-4
```

7.7. Magic Device (Simulation Only)

7.7.1. Overview

The Magic Device provides a simple interface for simulation control and console I/O without requiring a real UART.

> This peripheral is intended for simulation and testing only. It is not synthesizable for FPGA/ASIC implementation.

Table 31. Magic Device Features

Feature	Specification
Base Address	0xFFFF_0000
Size	64KB
Functionality	Console character output, simulation exit control

7.7.2. Register Map

Table 32. Magic Device Registers

Offset	Name	Access	Description
0xFFF0	EXIT	W/O	Write exit code to terminate simulation Value 1 = PASS (exit 0), Value ≠1 = FAIL (exit code N)
0xFFF4	CONSOLE	W/O	Write character (low byte) to simulation console

7.7.3. Exit Code Encoding

Magic device uses HTIF (Host-Target Interface) encoding for exit codes:

```
Pass:     write 1
Fail:     write ((exit_code << 1) | 1)
```

Example:

```
volatile uint32_t *magic_exit = (uint32_t *)0xFFFFFFF0;
```

```c
volatile uint32_t *magic_console = (uint32_t *)0xFFFFFFF4;

// Print to console
*magic_console = 'H';
*magic_console = 'i';
*magic_console = '\n';

// Exit with success
*magic_exit = 1;  // Simulation terminates with exit(0)
```

Chapter 8. AXI Interconnect and Bus Protocol

8.1. AXI4-Lite Overview

The SoC uses AXI4-Lite for peripheral communication and AXI4 for high-bandwidth memory access.

Table 33. AXI Protocol Summary

Feature	Specification
Protocol Version	AXI4 (AMBA 4 specification)
Peripheral Protocol	AXI4-Lite (32-bit address, 32-bit data)
Memory Protocol	AXI4 (32-bit address, 32-bit data, burst support)
Interconnect Topology	1-to-8 crossbar (single master, 8 slaves)
Outstanding Transactions	Supported via AXI ID mechanism (ID width = 4 bits, max 16 outstanding)
Burst Types	INCR (incrementing), WRAP (wrapping, for I-cache)
Response Codes	OKAY (00), SLVERR (10), DECERR (11)

8.2. Crossbar Architecture

The AXI crossbar (`axi_xbar`) routes transactions from the core to appropriate slave devices:

- **Master Port**: Connected to memory arbiter (combines I-fetch + D-access paths)
- **Slave Ports**: 8 devices (RAM, CLINT, PLIC, UART, I2C, SPI, DMA, Magic)

8.2.1. Address Decode Logic

Decoding is priority-based (higher-numbered slaves checked first):

```
if (addr[31:16] == 16'hFFFF)       → Slave 7 (Magic)
else if (addr[31:12] == 20'h2003_0) → Slave 6 (DMA)
else if (addr[31:16] == 16'h2002)  → Slave 5 (SPI)
else if (addr[31:16] == 16'h2001)  → Slave 4 (I2C)
else if (addr[31:16] == 16'h2000)  → Slave 3 (UART)
else if (addr[31:26] == 6'h03)     → Slave 2 (PLIC)
else if (addr[31:20] == 12'h020)   → Slave 1 (CLINT)
else if (addr[31:21] == 11'h400)   → Slave 0 (RAM)
else                               → SLVERR (unmapped)
```

8.2.2. Error Responses

- **Unmapped address read**: Crossbar returns `SLVERR` response with data = 0xDEADBEEF
- **Unmapped address write**: Crossbar returns `OKAY` response (write silently dropped)

8.3. AXI4 Memory Interface

The external RAM interface uses full AXI4 with burst support:

Table 34. AXI4 Memory Features

Feature	Specification
Data Width	32 bits
Burst Length	1-256 beats (ARLEN/AWLEN encoding)
Burst Types	INCR (normal access), WRAP (I-cache line fills)
Burst Size	4 bytes per beat (ARSIZE=3'b010)
Address Alignment	4-byte aligned
Write Strobes	4 bits (byte-level granularity)

8.3.1. I-Cache Burst Transactions

The instruction cache uses WRAP bursts for critical-word-first cache-line fills:

- Burst type: `ARBURST = 2'b10` (WRAP)
- Burst length: `ARLEN = (line_size / 4) - 1` (e.g., 7 for 32-byte line)
- Starting address: Critical word address (PC)
- Wrap boundary: Cache-line boundary

Example: Fetch from address 0x80000014 with 32-byte cache line:

```
ARADDR = 0x80000014
ARLEN = 7           // 8 beats
ARBURST = WRAP
Beats returned:
  0: 0x80000014 (critical word, returned first)
  1: 0x80000018
  2: 0x8000001C
  3: 0x80000000 (wrap to line boundary)
  4: 0x80000004
  5: 0x80000008
  6: 0x8000000C
  7: 0x80000010
```

8.3.2. Data Access Transactions

Data loads/stores use single-beat INCR bursts:

- Burst type: `ARBURST = 2'b01` (INCR)
- Burst length: `ARLEN = 0` (single beat)
- Write strobes: Indicate which bytes are valid (1, 2, or 4 bytes)

8.4. Bus Performance

Typical AXI transaction latencies:

Table 35. AXI Transaction Latencies

Transaction Type	Latency	Notes
Peripheral read (hit)	2-3 cycles	1 cycle AR, 1-2 cycles R
Peripheral write	1-2 cycles	1 cycle AW+W, write buffered
RAM read (no wait)	3 cycles	1 cycle AR, 2 cycles pipeline, 1 cycle R
RAM write (no wait)	2 cycles	1 cycle AW+W, 1 cycle B
I-cache line fill	10-20 cycles	8-beat WRAP burst + arbitration

Chapter 9. Electrical and Timing Characteristics

9.1. Clock and Reset

Table 36. Clock and Reset Specifications

Parameter	Value	Description
clk	Input	System clock (100 MHz nominal)
CLK_FREQ	100_000_000	Default clock frequency parameter (Hz)
rst_n	Active-low	Asynchronous reset, must be asserted for ≥2 clock cycles
Reset Recovery	2 cycles	Minimum cycles after rst_n de-assertion before operation

9.2. Timing Parameters

Table 37. Timing Characteristics (100 MHz, Tclk=10ns)

Parameter	Min	Max	Description
Setup Time (AXI signals)	2.0ns	-	Input setup before clock edge
Hold Time (AXI signals)	1.0ns	-	Input hold after clock edge
Clock-to-Output (AXI signals)	-	8.0ns	Output valid after clock edge
I-Cache Lookup Latency	1 cycle	1 cycle	Hit latency (registered output)
I-Cache Miss Penalty	10 cycles	50 cycles	Depends on memory latency
Load-to-Use Latency	2 cycles	2 cycles	Pipeline load latency
Branch Mispredict Penalty	2 cycles	2 cycles	Flush IF and ID stages (2 bubbles)
Interrupt Latency	3 cycles	10 cycles	From assertion to first trap instruction

9.3. Power Consumption (Estimated)

Power estimates are for Xilinx 7-series FPGA implementation at 100 MHz, 1.0V core voltage. Actual values depend on technology node, activity factor, and synthesis options.

Table 38. Power Consumption Estimates

Component	Power	Notes
Processor Core	~15 mW	Without I-cache, typical activity
I-Cache (4KB, 2-way)	~10 mW	SRAM power dominant
AXI Interconnect	~5 mW	Crossbar + arbiters

Component	Power	Notes
Peripherals (total)	~8 mW	UART, SPI, I2C, DMA idle
Total SoC	~40 mW	Typical workload @ 100 MHz

9.4. Environmental Specifications

Table 39. Operating Conditions (FPGA implementation)

Parameter	Value	Description
Operating Temperature	0°C to 85°C	Commercial temperature range
Core Voltage	1.0V ±5%	Xilinx 7-series VCCO
I/O Voltage	1.8V, 2.5V, 3.3V	Configurable per I/O bank

Chapter 10. Programming Guide

10.1. Boot Sequence

The processor boots from address `0x8000_0000` (RAM base) on reset:

1. Reset vector: PC = 0x8000_0000
2. Bootloader/firmware executes (typically copied to RAM by JTAG or external loader)
3. Initialize stack pointer
4. Initialize CSRs (mstatus, mtvec, mie)
5. Initialize peripherals (UART, timers, etc.)
6. Enable interrupts (clear mstatus.MIE)
7. Jump to main application

10.2. Memory Initialization

For simulation, the ELF loader (`elfloader.cpp`) loads program sections into RAM via DPI-C before simulation starts. For FPGA, use:

- **JTAG**: Load program via JTAG debugger (OpenOCD, etc.)
- **Boot ROM**: Store bootloader in on-chip ROM, copy program from external flash to RAM

10.3. Interrupt Handling

10.3.1. Setting Up Interrupts

```
#include "csr.h"
#include "rv_platform.h"
#include "rv_plic.h"
#include "rv_clint.h"

void setup_interrupts(void) {
    // Set trap vector (must be 4-byte aligned)
    write_csr(mtvec, (uint32_t)trap_entry);

    // Enable M-mode interrupts in mstatus
    set_csr(mstatus, MSTATUS_MIE);

    // Enable specific interrupt types in mie
    set_csr(mie, MIE_MEIE | MIE_MTIE | MIE_MSIE);

    // Configure PLIC
    rv_plic_set_priority(RV_PLIC_SRC_UART, 5);
    rv_plic_enable_interrupt(RV_PLIC_SRC_UART);
```

```
    rv_plic_set_threshold(0);

    // Configure CLINT timer for periodic interrupt (1ms)
    uint64_t mtime = rv_clint_read_mtime();
    rv_clint_write_mtimecmp(mtime + 100000);  // +1ms @ 100MHz
}
```

10.3.2. Trap Handler Example

```
void trap_handler(void) {
    uint32_t mcause = read_csr(mcause);
    uint32_t mepc = read_csr(mepc);

    if (mcause & 0x80000000) {
        // Interrupt
        uint32_t irq_code = mcause & 0x1F;
        switch (irq_code) {
        case 7:  // Timer interrupt
            handle_timer_interrupt();
            break;
        case 11: // External interrupt (PLIC)
            handle_external_interrupt();
            break;
        case 3:  // Software interrupt
            handle_software_interrupt();
            break;
        }
    } else {
        // Exception
        handle_exception(mcause, mepc);
    }
}

void handle_timer_interrupt(void) {
    // Advance mtimecmp for next interrupt
    uint64_t mtimecmp = rv_clint_read_mtimecmp();
    rv_clint_write_mtimecmp(mtimecmp + 100000);  // +1ms
}

void handle_external_interrupt(void) {
    // Claim interrupt from PLIC
    uint32_t source = rv_plic_claim_interrupt();

    switch (source) {
    case RV_PLIC_SRC_UART:
        uart_isr();
        break;
    case RV_PLIC_SRC_SPI:
        spi_isr();
```

```
        break;
    // ... handle other sources ...
    }

    // Complete interrupt
    rv_plic_complete_interrupt(source);
}
```

10.4. Peripheral Usage Examples

10.4.1. UART Echo Program

```
#include "rv_uart.h"

void uart_echo(void) {
    rv_uart_init(115200);  // Configure baud rate

    while (1) {
        if (rv_uart_rx_ready()) {
            uint8_t c = rv_uart_getc();
            rv_uart_putc(c);  // Echo character
        }
    }
}
```

10.4.2. SPI Flash Read

```
#include "rv_spi.h"

#define SPI_CMD_READ 0x03

void spi_read_flash(uint32_t addr, uint8_t *buf, size_t len) {
    rv_spi_init(1000000, RV_SPI_MODE0);  // 1 MHz, Mode 0

    rv_spi_select(0);  // Assert CS[0]

    // Send READ command + 24-bit address
    rv_spi_transfer(SPI_CMD_READ);
    rv_spi_transfer((addr >> 16) & 0xFF);
    rv_spi_transfer((addr >> 8) & 0xFF);
    rv_spi_transfer(addr & 0xFF);

    // Read data
    for (size_t i = 0; i < len; i++) {
        buf[i] = rv_spi_transfer(0x00);
    }
```

```
    rv_spi_deselect(0);  // De-assert CS[0]
}
```

10.4.3. I2C EEPROM Write

```
#include "rv_i2c.h"

#define EEPROM_ADDR 0x50  // 7-bit I2C address

void i2c_eeprom_write(uint8_t mem_addr, uint8_t data) {
    rv_i2c_init(100000);  // 100 kHz

    rv_i2c_start();
    rv_i2c_write(EEPROM_ADDR << 1);  // Address + W bit
    rv_i2c_write(mem_addr);           // Memory address
    rv_i2c_write(data);               // Data byte
    rv_i2c_stop();

    // Wait for write cycle to complete (typical 5ms)
    for (volatile int i = 0; i < 500000; i++);
}
```

10.4.4. DMA Memory Copy

```
#include "rv_dma.h"

void dma_memcpy(void *dst, const void *src, size_t len) {
    // Configure channel 0 for 1D transfer
    rv_dma_channel_config(0, (uint32_t)src, (uint32_t)dst, len,
                          RV_DMA_CTRL_MODE_1D |
                          RV_DMA_CTRL_SRC_INC |
                          RV_DMA_CTRL_DST_INC);

    // Enable interrupt
    rv_dma_channel_enable_interrupt(0);

    // Start transfer
    rv_dma_channel_start(0);

    // Wait for completion (or use interrupt)
    while (rv_dma_channel_busy(0));

    // Check for errors
    if (rv_dma_channel_error(0)) {
        // Handle error
    }
}
```

10.5. Performance Optimization Tips

1. **Enable I-Cache**: Set `ICACHE_EN=1` parameter for significant speedup (~2-5× depending on workload)
2. **Increase Buffer Depths**: Larger `IB_DEPTH` and `SB_DEPTH` improve throughput in memory-intensive code
3. **Use DMA for Bulk Transfers**: Offload large memory copies to DMA controller
4. **Minimize Load-Use Dependencies**: Reorder instructions to avoid pipeline stalls
5. **Use Fast Multiply/Divide**: Set `FAST_MUL=1` and `FAST_DIV=1` for single-cycle operations
6. **Align Data Structures**: 4-byte alignment avoids misaligned access penalties
7. **Batch Peripheral I/O**: Use FIFOs to buffer multiple bytes before checking status

10.6. Debugging

10.6.1. Simulation Debug

Use the magic device for quick debugging:

```
#define debug_printf(fmt, ...) \
    do { \
        char buf[128]; \
        snprintf(buf, sizeof(buf), fmt, ##__VA_ARGS__); \
        for (char *p = buf; *p; p++) \
            RV_MAGIC_CONSOLE = *p; \
    } while(0)

debug_printf("Checkpoint: pc=0x%08x, x10=%d\n", read_csr(mepc), x10);
```

10.6.2. Waveform Analysis

Generate VCD waveform dumps for RTL simulation:

```
make WAVE=1 rtl-simple
gtkwave build/tb_rv32_soc.vcd
```

Key signals to observe:

- **Pipeline stages**: `rv32_core.pc_if`, `state_id`, `alu_result_ex`
- **Memory transactions**: `imem_req_valid`, `dmem_resp_valid`, `axi_xbar` signals
- **Interrupts**: `timer_irq`, `external_irq`, `trap_entry`

10.6.3. Instruction Trace Comparison

Compare RTL and software simulator traces to debug mismatches:

```
make compare-simple
diff sim_trace.txt rtl_trace.txt
```

The trace format is:

```
<cycle> <pc> <instruction> <disassembly>
```

Chapter 11. Conclusion

This datasheet provides a complete reference for the RV32IMA SoC, covering architecture, memory map, peripheral specifications, electrical characteristics, and programming guidelines. For additional information, refer to:

- **RISC-V Specifications**: https://riscv.org/technical/specifications/
- **AMBA AXI4 Specification**: https://developer.arm.com/documentation/ihi0022/latest/
- **Project Repository**: https://github.com/YourOrg/rv32 (example)

For questions or support, contact: support@example.com

Appendix A: Appendix A: Register Quick Reference

A.1. CLINT Registers

Address	Name	Access	Reset Value
0x0200_0000	MSIP	R/W	0x0000_0000
0x0200_4000	MTIMECMP_LO	R/W	0xFFFF_FFFF
0x0200_4004	MTIMECMP_HI	R/W	0xFFFF_FFFF
0x0200_BFF8	MTIME_LO	R/W	0x0000_0000
0x0200_BFFC	MTIME_HI	R/W	0x0000_0000

A.2. PLIC Registers

Address	Name	Access	Reset Value
0x0C00_0000	PRIORITY[0]	R/O	0x0000_0000
0x0C00_0004-0x0C00_007C	PRIORITY[1..31]	R/W	0x0000_0000
0x0C00_1000	PENDING	R/O	0x0000_0000
0x0C00_2000	ENABLE	R/W	0x0000_0000
0x0C20_0000	THRESHOLD	R/W	0x0000_0000
0x0C20_0004	CLAIM/COMPLETE	R/W	0x0000_0000

A.3. UART Registers

Address	Name	Access	Reset Value
0x2000_0000	DATA	R/W	0x0000_0000
0x2000_0004	STATUS	R/O	0x0000_0002
0x2000_0008	IE	R/W	0x0000_0000
0x2000_000C	IS	R/O	0x0000_0000
0x2000_0010	LEVEL	R/O	0x0000_0000
0x2000_0014	CTRL	R/W	0x0000_0000

A.4. SPI Registers

Address	Name	Access	Reset Value
0x2002_0000	CTRL	R/W	0x0000_00F0
0x2002_0004	DIV	R/W	0x0000_0000
0x2002_0008	TX	W/O	-
0x2002_000C	RX	R/O	0x0000_0000
0x2002_0010	STATUS	R/O	0x0000_000A
0x2002_0014	IE	R/W	0x0000_0000
0x2002_0018	IS	R/O	0x0000_0000

A.5. I2C Registers

Address	Name	Access	Reset Value
0x2001_0000	CTRL	R/W	0x0000_0000
0x2001_0004	DIV	R/W	0x0000_0000
0x2001_0008	TX	W/O	-
0x2001_000C	RX	R/O	0x0000_0000
0x2001_0010	STATUS	R/O	0x0000_0002
0x2001_0014	IE	R/W	0x0000_0000
0x2001_0018	IS	R/O	0x0000_0000

A.6. DMA Global Registers

Address	Name	Access	Reset Value
0x2003_0F00	IRQ_STAT	R/W1C	0x0000_0000
0x2003_0F04	IRQ_EN	R/W	0x0000_0000
0x2003_0F08	DMA_ID	R/O	0xD4A0_0100
0x2003_0F10	PERF_CTRL	R/W	0x0000_0000
0x2003_0F14	PERF_CYCLES	R/O	0x0000_0000
0x2003_0F18	PERF_RD_BYTES	R/O	0x0000_0000
0x2003_0F1C	PERF_WR_BYTES	R/O	0x0000_0000

Appendix B: Appendix B: Interrupt Vector Table

Table 40. Machine-Mode Interrupt Vectors

Code	Type	Description	Source
0	Exception	Instruction address misaligned	Core
1	Exception	Instruction access fault	I-fetch AXI error
2	Exception	Illegal instruction	Core
3	Exception	Breakpoint	EBREAK
4	Exception	Load address misaligned	Core
5	Exception	Load access fault	Load AXI error
6	Exception	Store/AMO address misaligned	Core
7	Exception	Store/AMO access fault	Store AXI error
11	Exception	Environment call from M-mode	ECALL
0x8000_0003	Interrupt	Machine software interrupt	CLINT MSIP
0x8000_0007	Interrupt	Machine timer interrupt	CLINT timer
0x8000_000B	Interrupt	Machine external interrupt	PLIC

mcause[31]=1 indicates interrupt; mcause[30:0] contains interrupt code.

Appendix C: Appendix C: Pin Descriptions

C.1. Clock and Reset

Signal	I/O	Width	Description
clk	Input	1	System clock (100 MHz nominal)
rst_n	Input	1	Asynchronous active-low reset

C.2. UART Interface

Signal	I/O	Width	Description
uart_tx	Output	1	UART transmit data
uart_rx	Input	1	UART receive data

C.3. SPI Interface

Signal	I/O	Width	Description
spi_sclk	Output	1	SPI serial clock
spi_mosi	Output	1	SPI Master Out, Slave In
spi_miso	Input	1	SPI Master In, Slave Out
spi_cs_n	Output	4	SPI chip select (active-low)

C.4. I2C Interface

Signal	I/O	Width	Description
i2c_scl_o	Output	1	I2C clock output (0=pull low)
i2c_scl_i	Input	1	I2C clock input (for stretching)
i2c_scl_oe	Output	1	I2C clock output enable (1=high-Z)
i2c_sda_o	Output	1	I2C data output (0=pull low)
i2c_sda_i	Input	1	I2C data input
i2c_sda_oe	Output	1	I2C data output enable (1=high-Z)

I2C requires external pull-up resistors (4.7kΩ typical) on SCL and SDA.

C.5. External AXI4 Memory Interface

Signal	I/O	Width	Description
m_axi_awaddr	Output	32	Write address
m_axi_awlen	Output	8	Write burst length (beats - 1)
m_axi_awburst	Output	2	Write burst type (INCR=01)
m_axi_awvalid	Output	1	Write address valid
m_axi_awready	Input	1	Write address ready
m_axi_wdata	Output	32	Write data
m_axi_wstrb	Output	4	Write byte strobes
m_axi_wlast	Output	1	Write last beat
m_axi_wvalid	Output	1	Write data valid
m_axi_wready	Input	1	Write data ready
m_axi_bresp	Input	2	Write response (OKAY=00)
m_axi_bvalid	Input	1	Write response valid
m_axi_bready	Output	1	Write response ready
m_axi_araddr	Output	32	Read address
m_axi_arlen	Output	8	Read burst length (beats - 1)
m_axi_arsize	Output	3	Read burst size (010=4 bytes)
m_axi_arburst	Output	2	Read burst type (WRAP=10, INCR=01)
m_axi_arvalid	Output	1	Read address valid
m_axi_arready	Input	1	Read address ready
m_axi_rdata	Input	32	Read data
m_axi_rresp	Input	2	Read response (OKAY=00)
m_axi_rlast	Input	1	Read last beat
m_axi_rvalid	Input	1	Read data valid
m_axi_rready	Output	1	Read data ready

Appendix D: Appendix D: Revision History

Rev	Date	Author	Description
1.0	2026-02-25	RV32 Dev Team	Initial release